



Figure 2: PENTESTGPT v2 architecture. The TDA-EGATS Planner addresses Type B failures through difficulty-aware tree search with Upper Confidence Bound (UCB) selection, TDI-guided mode switching, and evidence-based pruning. The Tool & Skill Layer addresses Type A failures through typed tool interfaces and RAG-augmented knowledge. The Memory Subsystem maintains structured state and enables selective context injection based on tree position.

but does not assess path complexity. TermiAgent’s memory tree improves context management but does not inform exploration-exploitation decisions. None of these systems can answer the question that matters most: *is this path worth pursuing?*

3.3.2 Design Implications

Our analysis points to a two-part strategy for advancing LLM-based penetration testing. *Eliminating Type A failures* requires comprehensive tool interfaces with typed schemas, RAG systems for exploit documentation and Common Vulnerabilities and Exposures (CVE) databases, and standardized execution environments. This is tedious engineering work, but it produces predictable returns: each tool added directly enables new attack capabilities.

Addressing Type B failures requires a different approach: real-time difficulty estimation, principled exploration-exploitation decisions guided by the estimates, active pruning of intractable branches to prevent search collapse, and state maintenance external to conversation context to prevent information loss. These requirements suggest tree-based search algorithms to maintain state explicitly rather than relying on LLM’s context window.

Neither approach alone is sufficient. Capability engineering yields strong short-horizon performance but fails on complex tasks where navigation becomes the bottleneck. Planning innovation without adequate tooling produces agents that reason well but cannot execute. Effective systems must address both failure categories simultaneously, and in particular, agents need the ability to assess task difficulty in real time to avoid exploration-exploitation imbalance and chain failures.

4 Design of PENTESTGPT v2

4.1 Overview

We present PENTESTGPT v2, designed around the analysis in §3.3 to address both failure categories through dedicated architectural components. Figure 2 provides its architectural overview. PENTESTGPT v2 is a *single-agent* system that communicates with the environment consistently, operating over different components to complete penetration testing. It consists of the following modules: (1) A *Tool and Skill Layer* that eliminates Type A failures through structured tool interfaces and knowledge augmentation (§4.2). (2) A *Task Difficulty Assessment* (TDA) mechanism that estimates tractability in real time (§4.3), integrated into an *Evidence-Guided Attack Tree Search* (EGATS) algorithm that replaces the traditional PTT structure for exploration-exploitation decisions (§4.4). (3) A *Memory Subsystem* that maintains state across attack phases to prevent context forgetting (§4.5).

Given a target, PENTESTGPT v2 ❶ initializes an attack tree with the target as the root node. At each step, the EGATS planner consults the TDA module to select the current attack goal and updates the memory subsystem to preserve context. ❷ The selected goal is translated into concrete actions via the Tool and Skill Layer, and ❸ the resulting commands are executed in the test environment. ❹ Execution results are parsed and incorporated back into the attack tree and memory state, feeding into subsequent planning iterations until the penetration testing process terminates. Below we detail each component.

4.2 Tool and Skill Layer

Type A failures arise not from fundamental capability limitations, but from inconsistent tool usage: LLMs invoke security tools with incorrect parameters, misparse outputs, or lack domain knowledge about tool capabilities. Rather than proposing novel techniques, the Tool and Skill Layer represents

careful engineering to ensure LLM agents interact with security tools consistently and reliably. We build on established concepts of Agent Skills [4] from Anthropic (typed interfaces, skill composition, and retrieval-augmented generation), adapting them to penetration testing where tool reliability directly determines attack success.

Typed Tool Interfaces. Each security tool is exposed through a typed interface specifying input schema (parameters with types, defaults, and validation rules), output schema (structured representation parsed from command output), and pre-/postconditions (required state before invocation and expected effects after completion). The LLM receives explicit documentation rather than relying on parametric knowledge. Input validation catches errors before execution, and structured outputs eliminate parsing ambiguity. We implement interfaces for 38 tools across six categories: reconnaissance, web exploitation, network exploitation, credential attacks, Active Directory attacks, and privilege escalation. Appendix B provides the complete tool list we integrate.

Skill Composition. Beyond individual tools, *skills* compose multiple tool invocations into higher-level attack capabilities that encode expert knowledge about common attack patterns. Skills provide fallback logic so that when a preferred tool fails, the system can try alternatives automatically. They also aggregate results from multiple tools into coherent findings and encode multi-step attack patterns that reflect how human testers chain operations.

Knowledge Augmentation. The layer integrates a RAG system containing tool documentation, an exploit database (CVE descriptions indexed by service version), and attack playbooks (step-by-step procedures for common patterns such as Kerberoasting, AS-REP roasting, and pass-the-hash). The knowledge base contains only generic attack techniques from public security resources (MITRE ATT&CK, OWASP, tool documentation); it excludes CTF writeups, HTB walkthroughs, or benchmark-specific solutions to prevent data leakage in evaluation. When the agent encounters an unfamiliar service or vulnerability class, relevant documentation is retrieved and injected into context automatically.

These three mechanisms together provide a unified, reliable interface between LLM agents and security tools. None of these techniques is novel in isolation; their contribution lies in the combination, which minimizes tool invocation errors that otherwise cascade into attack failures. Our ablation study (§5.3) shows that this engineering effort yields substantial gains on capability-limited tasks: the Tool Layer alone improves XBOW completion by 14% (from 54% to 68%), allowing agents to focus their reasoning on the harder problems of planning and strategy.

4.3 Task Difficulty Assessment (TDA)

Our analysis in §3.3 identifies the inability to assess task difficulty as the root cause of Type B failures. Premature commit-

ment occurs because agents cannot estimate whether a path requires 3 or 30 steps. Exploration-exploitation imbalance occurs because agents have no metric for when reconnaissance is sufficient. Chain failures occur because agents cannot judge whether accumulated context is adequate for the current task.

Human penetration testers face the same problem: they do not know task difficulty *a priori*. Instead, they estimate difficulty from signals that accumulate during execution, such as the number of failed attempts on a path, the quality of evidence gathered so far, and intuitions about remaining work. An experienced tester who has tried five exploits without success knows to try a different approach; one who has confirmed a vulnerable service version knows to commit to exploitation. TDA operationalizes this reasoning for LLM agents through four measurable dimensions, with context window consumption added as a signal unique to language models.

4.3.1 TDA Dimensions

TDA computes difficulty along four dimensions grounded in quantities measurable during execution.

Horizon Estimation (H). We estimate the number of remaining steps to reach the goal from the current position, normalized across active branches. A pilot study on 50 traces from an independent GOAD deployment (using GPT-4o, separate from evaluation) shows that while absolute estimates have poor calibration (MAE of 4.2 steps), rank correlation is strong (Spearman’s $\rho = 0.71$, $p < 0.001$). The TDI formula therefore uses $\hat{H}$, the *normalized* horizon estimate (min-max scaled across active branches), converting absolute estimates into relative rankings where LLM judgment is reliable.

Historical Success Rate (S). The Laplace-smoothed success rate on the current branch captures learning from failed attempts. Low values indicate repeated failures, suggesting that the current path is likely intractable. This dimension directly addresses *premature commitment*: agents learn to abandon unproductive paths rather than persisting indefinitely.

Context Load (C). The fraction of context window consumed, directly measurable from token counts. LLM performance degrades as context fills: retrieval accuracy drops, earlier information is forgotten, and reasoning quality declines [18]. We define an *ideal working window* of 40% of the model’s context capacity, based on a controlled study showing consistent accuracy degradation beyond this point (94% $\rightarrow$ 78% at 60% load, 61% at 80%; see Appendix D.6). Beyond this threshold, context pruning becomes necessary to preserve reasoning quality. This dimension addresses *context forgetting*: by tracking context load, the system detects when accumulated history threatens to overwhelm the model’s effective memory.

Evidence Confidence (E). The mean confidence score across the path from root to current node, computed from evidence categories at each node. We assign scores based on evidence type: verified exploits and valid credentials receive 1.0, con-

firmed vulnerabilities with available exploits receive 0.8, plausible hypotheses (version-matched vulnerabilities, misconfigurations) receive 0.5, and speculative hypotheses receive 0.3. Tool outputs are parsed to determine evidence type: successful authentication or shell access indicates verified evidence, vulnerability scanner confirmations with CVE matches indicate confirmed vulnerabilities, and service version matches against exploit databases indicate plausible hypotheses. Appendix C details the complete scoring rubric. This dimension addresses *exploration-exploitation imbalance*: high confidence signals readiness to exploit, while low confidence signals the need for more reconnaissance.

4.3.2 Task Difficulty Index

TDA combines the above 4 dimensions into a Task Difficulty Index (TDI):

$$\text{TDI} = w_H \cdot \hat{H} + w_E \cdot (1 - E) + w_C \cdot C + w_S \cdot (1 - S) \quad (1)$$

where $\hat{H}$ is the normalized horizon estimate and all weights sum to 1. Higher TDI indicates greater difficulty. We set $w_H = w_E = 0.3$ and $w_C = w_S = 0.2$ based on grid search over a validation set of 30 execution traces from HTB machines not included in the PentestGPT benchmark (retired machines from 2022–2023, predating our evaluation set). We test 256 configurations with each weight in $\{0.1, 0.2, 0.3, 0.4\}$ constrained to sum to 1.0; task completion varies within $\pm 3\%$ across configurations where all weights remain in $[0.1, 0.4]$, indicating that the approach is not sensitive to precise weight selection.

TDI guides three operational decisions. *Mode selection*: high TDI ($> \theta_{\text{exploit}} = 0.6$) triggers reconnaissance (BFS) to gather more information before committing; low TDI ($< \theta_{\text{exploit}} = 0.3$) triggers exploitation (DFS). For intermediate values ($0.3 \leq \text{TDI} \leq 0.6$), the system invokes LLMDECIDE: the LLM receives the current node state, TDI value, and individual dimension scores (H, S, C, E), then selects between reconnaissance and exploitation with a brief justification. This design acknowledges that intermediate difficulty may warrant either approach depending on context the TDI formula cannot fully capture. For instance, a moderately difficult path with high evidence confidence may warrant exploitation, while one with low confidence benefits from further reconnaissance. *Branch prioritization*: TDI ranks paths beyond promise scores alone, since two branches with similar promise may differ substantially in tractability based on horizon and success history. *Pruning*: branches with persistently high TDI ($> \theta_{\text{prune}} = 0.8$) after $k_{\text{min}} = 3$ attempts are pruned to prevent the search from collapsing into unproductive regions. These thresholds are derived through grid search on the same validation set used for TDI weights. Appendix D presents sensitivity analysis showing robustness across threshold ranges.

Table 4: Search strategy comparison. EGATS is the only approach that combines external structure, evidence-based pruning, and TDA-guided mode selection.

Approach	Structure	Pruning	Difficulty	TDA
ReAct	None	–	–	–
PTT [8]	Tree (text)	Manual	–	–
PSM [32]	Finite state machine	–	–	–
PMT [20]	Tree	–	–	–
EGATS	Tree (ext.)	Evidence	✓	✓

4.4 Evidence-Guided Attack Tree Search (EGATS)

EGATS integrates TDA into a tree-based search framework, adapting Monte Carlo Tree Search (MCTS) [6, 16] to penetration testing. EGATS differs from standard MCTS in three ways: it explicitly separates reconnaissance (BFS) and exploitation (TDI-guided) phases, it replaces simulation-based value estimates with TDA-based difficulty assessment, and it prunes intractable branches based on evidence.

4.4.1 Attack Tree Structure

EGATS maintains an Attack Tree $\mathcal{T} = (V, E, \phi, \psi, \delta)$ where V contains nodes representing attack states, E contains edges representing actions, $\phi : V \rightarrow [0, 1]$ assigns promise scores, $\psi : V \rightarrow \mathcal{S}$ maps nodes to state snapshots, and $\delta : V \rightarrow [0, 1]$ assigns TDI scores. Nodes are categorized as *observation* (discovered facts), *hypothesis* (untested attack possibilities), or *action* (executed steps with outcomes).

The *promise score* $\phi(n)$ estimates the likelihood that node n leads to successful exploitation. For hypothesis nodes, promise is initialized via LLM assessment of vulnerability severity, exploit availability, and prerequisite satisfaction; the model estimates success probability given current evidence. For action nodes, promise is updated based on execution outcomes: successful actions propagate increased promise to ancestor nodes, while failures decrease promise along the path. After action a with outcome $o \in \{\text{success}, \text{partial}, \text{failure}\}$, we update $\phi(n) \leftarrow \alpha \cdot \phi(n) + (1 - \alpha) \cdot r(o)$ where $r(\text{success}) = 1.0$, $r(\text{partial}) = 0.5$, $r(\text{failure}) = 0.1$, and $\alpha = 0.7$ controls the learning rate. Through this backpropagation, branches with consistent successes accumulate high promise while repeatedly failing branches see diminishing scores.

Unlike PentestGPT’s text-based PTT, EGATS maintains structure externally via algorithmic operations, which prevents corruption and enables systematic search guidance. Table 4 compares EGATS with related approaches.

4.4.2 The EGATS Algorithm

Algorithm 1 presents the TDA-guided search procedure. SELECTNODE uses UCB to balance exploitation and explo-

Algorithm 1 TDA-Guided Attack Tree Search

Require: Target T , budget B

Ensure: Attack tree $\mathcal{T}$, compromised hosts C

```
1:  $\mathcal{T} \leftarrow \text{INITTREE}(T)$ 
2: while  $B > 0$  and not GOALREACHED do
3:    $n \leftarrow \text{SELECTNODE}(\mathcal{T})$   $\triangleright$  UCB selection
4:    $\text{TDI}_n \leftarrow \text{COMPUTETDI}(n)$ 
5:   if  $\text{TDI}_n > \theta_{\text{explore}}$  then
6:     EXECUTERECON( $n$ ); EXPANDTREE( $\mathcal{T}, n$ )
7:   else if  $\text{TDI}_n < \theta_{\text{exploit}}$  then
8:      $\text{result} \leftarrow \text{EXECUTEEXPLOIT}(n)$ 
9:     BACKPROPAGATEEVIDENCE( $\mathcal{T}, n, \text{result}$ )
10:    if  $\text{result.success}$  then SPAWNPIVOT( $\mathcal{T},$ 
11:       $\text{result.host}$ )
12:    end if
13:  else
14:    LLMDECIDE( $n, \text{TDI}_n$ )
15:  end if
16:  if  $\delta(n) > \theta_{\text{prune}}$  and  $N_n > k_{\text{min}}$  then
17:    PRUNEBRANCH( $\mathcal{T}, n$ )
18:  end if
19:   $B \leftarrow B - 1$ 
20: end while
```

ration:

$$\text{UCB}(n) = \phi(n) + c\sqrt{\frac{\ln N}{N_n}} - \lambda \cdot \delta(n) \quad (2)$$

where $\phi(n)$ is the promise score, N is total actions, N_n is actions on node n 's subtree, $c = \sqrt{2}$ is the exploration constant, and the $-\lambda \cdot \delta(n)$ term penalizes high-difficulty nodes ($\lambda = 0.5$, validated via grid search; see Appendix D).

After selection, EGATS computes TDI and switches between BFS (reconnaissance) and DFS (exploitation) based on the thresholds described above. Evidence backpropagates after each action, updating promise scores and TDI along affected paths. When exploitation succeeds, *pivot spawning* is triggered: the compromised host becomes a new subtree root, and discovered credentials propagate to relevant hypothesis nodes elsewhere in the tree.

Pruning removes branches when TDI exceeds 0.8 after three attempts, which prevents infinite loops on intractable paths. To avoid premature pruning, a credential propagation mechanism re-evaluates pruned branches when new credentials are discovered that may satisfy their preconditions.

4.5 Memory Subsystem

Long-context forgetting is a primary cause of Type B failures (§3.2). The Memory Subsystem addresses this with a hybrid architecture that separates persistent state from conversational context, and integrates with TDA via the context load dimension.

A *State Store* maintains a structured database of discovered facts independent of conversation context. The store tracks five entity types: hosts (IP addresses, hostnames, OS fingerprints), services (ports, versions, configurations), credentials (usernames, passwords, hashes, tickets), sessions (active shells, tunnels, pivots), and vulnerabilities (CVE identifiers, exploitation status, prerequisites). Each entry is timestamped and linked to its discovery node in the attack tree, which enables provenance tracking and ensures facts persist regardless of conversation length. The State Store also supports accurate TDA context load computation by providing ground truth about what information the agent “knows” versus what must be re-derived from context.

Selective context injection replaces full history maintenance. When operating on node n , context is assembled from: path context (the sequence of actions from root to n), a node state snapshot (complete state at n including all relevant entity relationships), target-relevant facts (entries from State Store pertaining to n 's target host or service), and sibling branch summaries (compressed representations of parallel exploration paths). As context load approaches the ideal working window threshold (40%), less-relevant context is progressively compressed using LLM-generated summaries. Beyond 70%, aggressive pruning removes older path segments while preserving findings to prevent performance degradation.

Branch summaries compress detailed execution history when switching branches. Each summary preserves the current status (active, pruned, completed), findings (discovered credentials, confirmed vulnerabilities), TDI at time of suspension, and recommended next actions. TDI is stored with each summary to inform revisit decisions: when new credentials are discovered elsewhere in the tree, branches with matching preconditions and previously high TDI are re-evaluated for potential reactivation.

5 Evaluation

We assess the performance of PENTESTGPT v2 through four research questions:

- **RQ1:** Does PENTESTGPT v2 outperform existing systems across different penetration testing scenarios?
- **RQ2:** What is the contribution of the each designed architectural component?
- **RQ3:** How does TDA-EGATS change the agent's attack strategy compared to prior approaches?
- **RQ4:** Can PENTESTGPT v2 be practically deployed for real-world penetration testing?

5.1 Experimental Setup

PENTESTGPT v2 is implemented in Python (~8,500 lines), with the Tool Layer, TDA-EGATS Planner, and Memory Subsystem as separate modules. The implementation is open-

Table 5: Performance comparison across systems, models, and benchmarks. Each model column is split into non-thinking (–) and thinking (T) modes. XBOW: task completion (%); PentestGPT Benchmark: machines rooted (/13); GOAD: hosts compromised (/5). Best results per column in **bold**. All results report mean across 3 trials; variance $\pm 2\text{--}3\%$ on XBOW, ± 1 machine on PentestGPT-Ben.

System	XBOW (104 tasks)						PentestGPT-Ben (13 machines)						GOAD (5 hosts)					
	GPT-5.2		Opus 4.5		Gemini 3		GPT-5.2		Opus 4.5		Gemini 3		GPT-5.2		Opus 4.5		Gemini 3	
	–	T	–	T	–	T	–	T	–	T	–	T	–	T	–	T	–	T
PentestGPT	45	53	47	54	41	48	7	8	6	7	6	7	1	1	1	2	1	1
AutoPT	43	50	44	51	38	45	6	7	7	8	5	6	1	1	0	1	1	1
PentestAgent	52	61	54	60	46	54	8	9	7	9	7	8	1	2	2	2	1	1
VulnBot	48	56	50	58	43	51	8	9	8	9	6	8	2	2	1	2	1	2
PENTESTGPT v2	76	85	81	91	76	79	11	12	10	12	10	11	3	4	3	4	3	3

sourced [3]. Following the evaluation methodology in Section 3.1, we evaluate PENTESTGPT v2 on three benchmarks of increasing complexity. **XBOW** [2] comprises 104 CTF-style web security challenges covering SQL injection, XSS, authentication bypass, and file inclusion; these short-horizon tasks isolate Type A failures where tool usage determines success. The **PentestGPT Benchmark** [8] consists of 13 machines from HTB and VulnHub, requiring end-to-end penetration testing from reconnaissance through privilege escalation to root access. Difficulty ranges from Easy to Hard, with 9–22 subtasks per machine, representing realistic scenarios that demand multi-step attack chains. **GOAD** [25] provides a 5-host multi-domain Active Directory environment requiring credential harvesting, Kerberoasting, lateral movement, and domain escalation, complex enterprise scenarios dominated by Type B failures.

We compare against four baseline systems: PentestGPT v1.0 [8], AutoPT [32], PentestAgent [30], and VulnBot [17]. We exclude Cochise [12] from this comparison because its AD-specialized architecture creates an uneven evaluation as shown in Section 3.2. Baseline systems use their original tool invocation mechanisms to reflect realistic deployment comparisons; reported improvements therefore reflect both tool integration and architectural contributions. To isolate architectural contributions from model capabilities, all systems are evaluated with three frontier models: GPT-5.2, Claude-Opus-4.5, and Gemini-3.0-Pro. We select these models for two reasons: (1) they represent state-of-the-art capabilities at the time of evaluation, and (2) all three support toggling between standard and thinking modes, enabling controlled comparison of extended reasoning effects. We report task completion rate, subtask progress, and exploration metrics including branch diversity, backtrack frequency, and time-to-pivot. We report mean performance across trials with standard deviation where variance is meaningful; for discrete outcomes (machines rooted, hosts compromised), we report best-of-three following prior work [8, 9] since standard deviation on small integers provides limited insight. For XBOW’s continuous completion rates, we report both headline best-of-three results

and trial statistics (μ : mean, σ : standard deviation across the three trials) to characterize variance. In total, we conduct 5 systems $\times$ 118 evaluation units $\times$ 6 model configurations $\times$ 3 trials, yielding 10,620 evaluation runs at an estimated cost of \$2,760 USD in API tokens (Table 8 reports PENTESTGPT v2-specific costs).

5.2 RQ1: Overall Performance

Table 5 shows the performance comparison across all system-model-benchmark combinations, with consistent patterns that align with our Type A/B failure framework.

On XBOW, PENTESTGPT v2 achieves 91% task completion (best-of-3; $\mu=89\%$, $\sigma=2.1\%$) with Opus 4.5 thinking mode, a 49% relative improvement over the best baseline (PentestAgent at 61%, $\mu=59\%$, $\sigma=1.8\%$). With GPT-5.2 thinking, PENTESTGPT v2 achieves 85% ($\mu=83\%$, $\sigma=2.4\%$) compared to 61% for PentestAgent. Even comparing means, the gap (89% vs. 59%) exceeds 15 standard deviations, confirming robust architectural differences: the Tool Layer eliminates Type A failures while TDA-EGATS prevents trial-and-error loops that consume baseline attempts. Thinking mode provides 6–10 point improvements across all systems and configurations but does not close the architectural gap.

The PentestGPT benchmark shows larger architectural differences. PENTESTGPT v2 roots 12 of 13 machines with both GPT-5.2 and Opus 4.5 thinking (consistent across all three trials), compared to 9 for the best baseline (VulnBot), a 33% relative improvement. PENTESTGPT v2 solves both Hard-rated machines (Joker and Falafel) where baseline systems became “stuck at initial steps,” and also completes machines that require non-obvious attack chains. The improvement concentrates in machines requiring non-linear attack paths: while baseline PTT structures lead to premature commitment on initial hypotheses, TDA-EGATS enables strategic backtracking when evidence confidence drops, allowing the agent to discover alternative attack vectors. Thinking mode amplifies architectural differences: PENTESTGPT v2 gains 1–2 machines from thinking, achieving near-complete coverage,